

GRADUAÇÃO EM CIÊNCIA DE DADOS · AULA 01

Fundamentos da Infraestrutura em Projetos de Big Data

Da coleta ao consumo: como sustentar dados em escala

Objetivos de aprendizagem e roteiro

AO FINAL DA AULA, VOCÊ SERÁ CAPAZ DE

- ✓ Caracterizar um problema de Big Data pelos 5 Vs
- ✓ Descrever as camadas de uma plataforma de dados
- ✓ Comparar batch × streaming e DW × Lake × Lakehouse
- ✓ Diferenciar on-premises, IaaS, PaaS e SaaS
- ✓ Avaliar desempenho, custo e conformidade (LGPD) na escolha da infraestrutura

ROTEIRO

- 1 **Fundamentos** — O que muda quando o dado cresce
- 2 **Ingestão** — Coleta, batch, streaming e CDC
- 3 **Armazenamento** — Lake, Warehouse, formatos e custo
- 4 **Processamento** — Cluster, Spark e containers
- 5 **Nuvem** — On-premises, IaaS, PaaS e SaaS
- 6 **Operação** — Rede, segurança, LGPD e custos

Seis quizzes ao longo da aula marcam os pontos de verificação.

O que caracteriza um problema de Big Data



Volume

Escala que ultrapassa a capacidade de uma única máquina.

Ex.: 50 TB de logs por mês



Velocidade

Ritmo de geração e prazo aceitável de disponibilização.

Ex.: clique tratado em 1 s



Variedade

Dados estruturados, semiestruturados e não estruturados.

Ex.: SQL, JSON, vídeo, áudio



Veracidade

Confiabilidade, ruído, lacunas e viés dos dados.

Ex.: sensor descalibrado



Valor

Retorno que justifica o custo da infraestrutura.

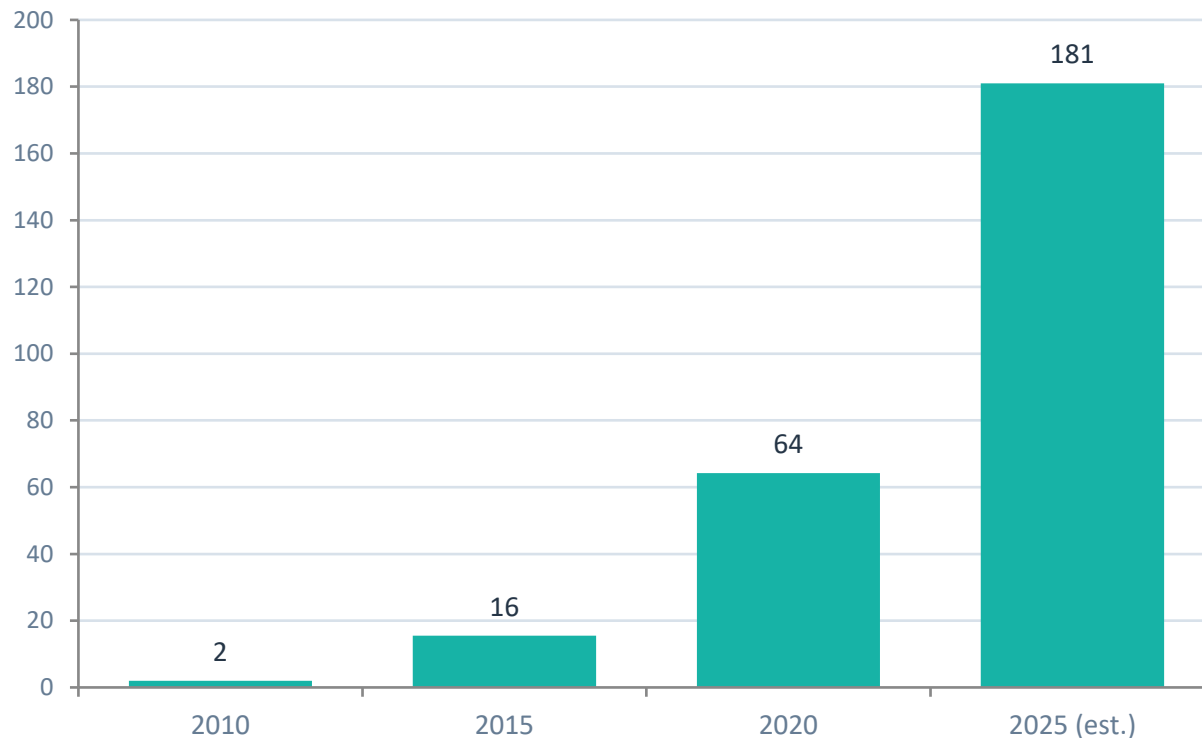
Ex.: churn evitado × custo do cluster



Regra prática: se um servidor robusto com um banco relacional resolve com folga, você ainda não tem um problema de Big Data — tem um problema de dados. Infraestrutura distribuída cobra em complexidade o que entrega em escala.

Por que a infraestrutura vira o gargalo

Dados criados e replicados no mundo (ZB)



Estimativas IDC / Statista — ordem de grandeza, não valor exato.



O crescimento não é linear

Dobrar o volume pode multiplicar custo e tempo de resposta quando a arquitetura não escala horizontalmente.



O gargalo se desloca

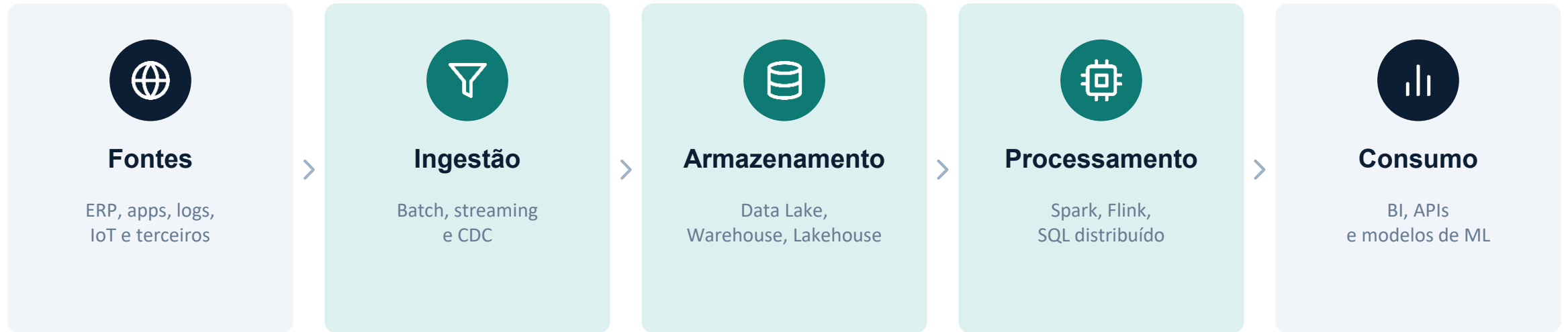
Resolvido o processamento, o limite passa para a E/S de disco; resolvido o disco, passa para a rede.



A decisão dura anos

Escolha de infraestrutura vira custo recorrente e dívida técnica difícil de reverter no meio do projeto.

Anatomia de uma plataforma de dados



CAMADAS TRANSVERSAIS — PRESENTES EM TODAS AS ETAPAS



Orquestração

Airflow, agendamento e retentativa



Segurança

IAM, criptografia e LGPD



Observabilidade

Logs, métricas e qualidade



Custos

FinOps e dimensionamento

Cada camada tem seu próprio gargalo, sua tecnologia e seu custo — e falha de um jeito diferente.

Camada 1 — Coleta: onde os dados nascem



Onde os dados nascem

Sistemas internos (ERP, CRM, e-commerce), logs de aplicação, sensores IoT, parceiros comerciais e bases públicas.



Em que formato chegam

Estruturados (tabelas), semiestruturados (JSON, XML, logs) e não estruturados (texto, imagem, áudio, vídeo).



O que quebra na prática

Mudança de esquema na origem, eventos duplicados ou fora de ordem, picos de carga, falha de rede e credenciais expiradas.

Princípio da camada bruta: guarde o dado original, sem transformação. Regra de negócio muda, transformação se refaz — coleta perdida, não se recupera.



Batch × streaming: a decisão que define o custo



Processamento em lote (batch)

LATÊNCIA

De horas a alguns minutos

UNIDADE DE TRABALHO Conjunto de arquivos ou partições

CUSTO

Menor por GB: o cluster sobe, roda e desliga

QUANDO USAR

Relatórios, fechamento contábil, cargas históricas e treino de modelos



Processamento em fluxo (streaming)

LATÊNCIA

De segundos a milissegundos

UNIDADE DE TRABALHO Evento individual ou micro-lote

CUSTO

Maior: infraestrutura permanentemente ativa

QUANDO USAR

Antifraude, monitoramento de operação, alertas e personalização ao vivo

Arquiteturas híbridas: Lambda mantém dois caminhos (lote + tempo real) e concilia os resultados; Kappa usa só streaming e reprocessa a partir do log de eventos.

O que se usa: ferramentas da camada de ingestão



Apache Kafka

Log distribuído de eventos: desacopla produtores de consumidores, retém e permite reprocessar.



Flink / Spark Streaming

Motores de processamento contínuo com janelas de tempo, estado e tolerância a falhas.



CDC (Debezium)

Captura mudanças lendo o log de transações do banco, sem sobrecarregar o sistema de origem.



NiFi / Airbyte

Conectores prontos para mover dados entre centenas de fontes e destinos com pouca codificação.



Apache Airflow

Orquestra o pipeline: dependências, agendamento, retentativas, alertas e histórico de execução.



Coletores de borda

Agentes leves (Fluent Bit, MQTT) para logs de servidores e dispositivos IoT com pouca banda.

Regra de leitura: cada ferramenta resolve um papel — transporte, processamento, captura, integração ou orquestração. Combiná-las mal cria pipelines frágeis.

Camada 2 — Hierarquia de acesso e custo

PIRÂMIDE DE ACESSO — QUANTO CUSTA ESPERAR

Memória (RAM)

~100 nanossegundos

SSD NVMe

~100 microssegundos

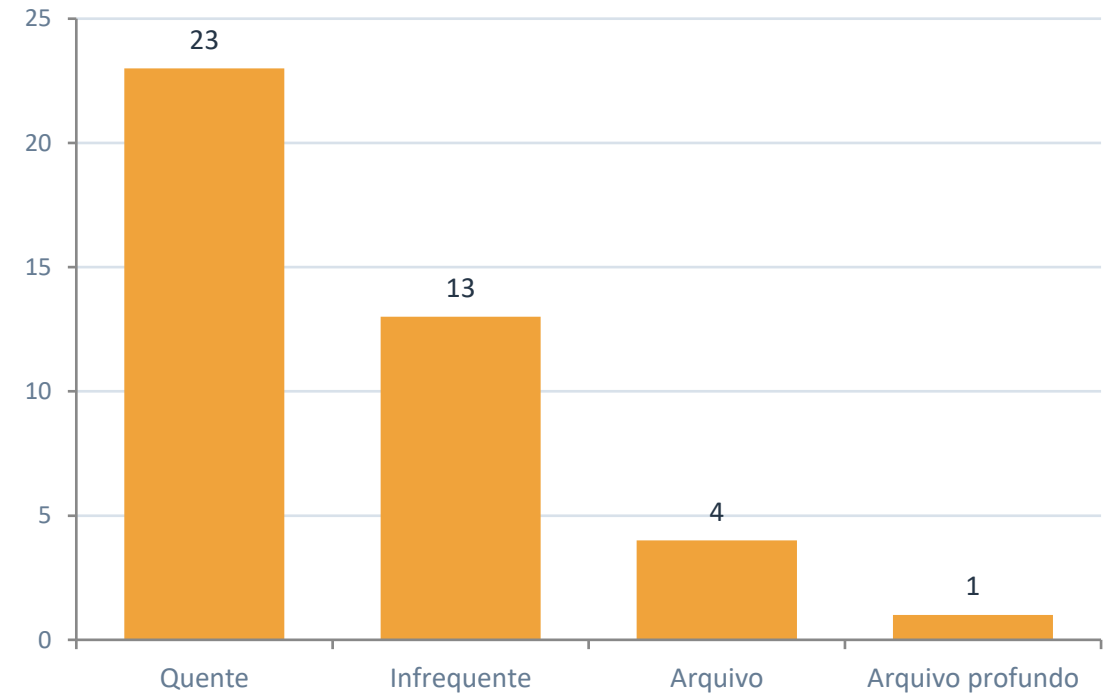
Disco magnético (HDD)

~10 milissegundos

Arquivo frio (fita, glacier)

minutos a horas

Custo do armazenamento de objetos (US\$/TB por mês)



Valores de referência de mercado — ordem de grandeza. Recuperar dado frio custa tempo e taxa de resgate.

Um TB quente custa cerca de 20× o mesmo TB em arquivo profundo. Definir o ciclo de vida do dado (tiering) é decisão de projeto, não detalhe operacional.

Data Warehouse × Data Lake × Lakehouse



Data Warehouse

O QUE É

Dados já tratados e modelados para consulta analítica

ESQUEMA

schema-on-write: valida na entrada

MELHOR PARA

BI, indicadores oficiais, séries confiáveis

RISCO

Rígido e caro para dado bruto ou não estruturado



Data Lake

O QUE É

Repositório de dados brutos em armazenamento de objetos

ESQUEMA

schema-on-read: interpreta na leitura

MELHOR PARA

Ciência de dados, dado não estruturado, exploração

RISCO

Virar "pântano de dados" sem catálogo e sem dono



Lakehouse

O QUE É

Camada transacional sobre os arquivos do lake

ESQUEMA

Híbrido, com evolução controlada

MELHOR PARA

BI e ML sobre a mesma cópia dos dados

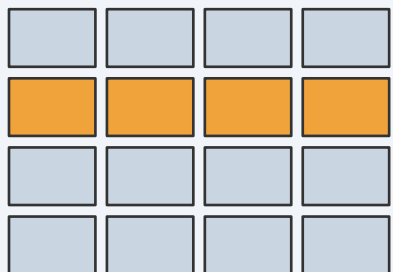
RISCO

Exige governança e disciplina de engenharia

Tendência de mercado: convergência. O lakehouse busca o custo do lake com as garantias transacionais do warehouse — e é o desenho padrão dos projetos novos.

Formatos importam: linha × coluna

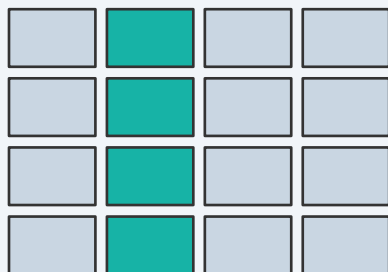
ORIENTADO A LINHA



CSV, JSON, Avro

Lê o registro inteiro, mesmo que a consulta use 2 campos.

ORIENTADO A COLUNA



Parquet, ORC

Lê apenas as colunas pedidas e comprime muito melhor.

Mesma consulta, menos dado lido: menos E/S, menos tempo, menos custo.

POR QUE COLUNAR GANHA

- Lê só as colunas da consulta: menos E/S
- Compressão muito maior: valores semelhantes agrupados
- Estatísticas por bloco permitem pular arquivos inteiros

FORMATOS DE TABELA: DELTA, ICEBERG, HUDI

- Transações ACID sobre arquivos no Data Lake
- Time travel: consultar o estado anterior da tabela
- Evolução de esquema sem reescrever o histórico

Trocar CSV por Parquet particionado costuma ser o ganho de desempenho mais barato de um projeto de dados.

Distribuir: replicação, particionamento e localidade



Replicação

O HDFS grava cada bloco em 3 nós por padrão (o diagrama ilustra fator 2). Custa espaço e compra disponibilidade.



Particionamento

Divide a tabela por chave (data, região, cliente). A consulta que filtra pela chave lê só a fatia relevante — partition pruning.



Localidade dos dados

Mover megabytes de código é barato; mover terabytes de dados, não. O motor tenta processar onde o dado já está.

Camada 3 — Escalar para cima ou para os lados



Vertical (scale up)

Uma máquina maior

VANTAGENS

- Simples: nenhuma mudança no código
- Sem coordenação nem rede entre nós
- Menos peças para operar e monitorar

LIMITES

- Existe um teto físico de CPU, RAM e disco
- Preço cresce mais rápido que a capacidade
- Ponto único de falha



Horizontal (scale out)

Muitas máquinas comuns

VANTAGENS

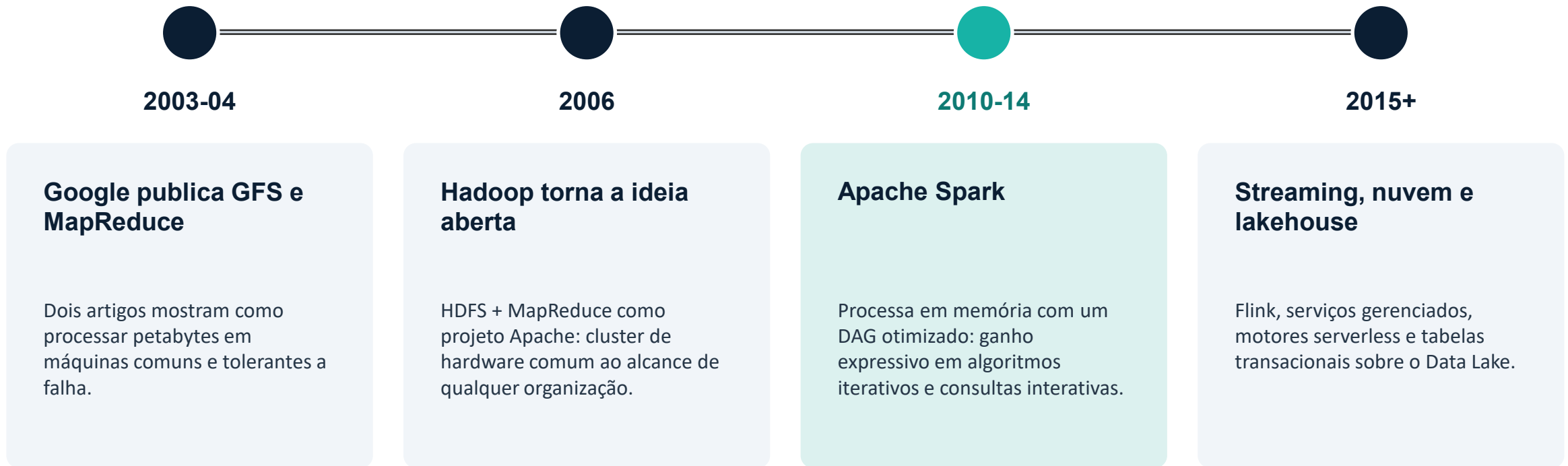
- Escala quase linear somando nós
- Tolerância a falha por replicação
- Hardware comum, mais barato por unidade

LIMITES

- Exige coordenação e código distribuído
- A rede vira gargalo (shuffle)
- Consistência e depuração mais difíceis

Em escala, falha é rotina e não exceção: com centenas de nós, algo quebra toda semana. O software precisa assumir isso — e é essa premissa que o Hadoop e o Spark trouxeram.

De MapReduce ao ecossistema atual



Por que o Spark é mais rápido em tarefas iterativas: em vez de gravar o resultado intermediário em disco a cada par map/reduce, ele mantém os dados em memória e otimiza o plano de execução inteiro (DAG) antes de rodar.

Containers: o ambiente também é infraestrutura



Docker · o container

- Empacota código, bibliotecas e sistema de arquivos em uma imagem versionada
- Isola dependências: dois jobs com versões diferentes de Python convivem
- Reprodutibilidade: a mesma imagem roda no notebook, no CI e no cluster
- Leve: compartilha o núcleo do sistema operacional, ao contrário da máquina virtual



Kubernetes · a orquestração

- Agenda containers nos nós conforme CPU e memória disponíveis
- Autoescala: sobe réplicas na carga alta e devolve recursos na carga baixa
- Autocorreção: recria o container que morreu e tira do tráfego o que falha
- Atualização gradual sem interromper o serviço em produção



No laboratório da disciplina: você vai empacotar o job de MapReduce em uma imagem Docker e executá-lo no GitHub Codespaces. É a mesma ideia usada em produção, só que em escala reduzida.

Camada 4 — Consumo: do dado ao insight



Preparação

Limpeza, integração e validação. Consome a maior parte do esforço analítico — sem qualidade, não existe modelo confiável.



Modelagem

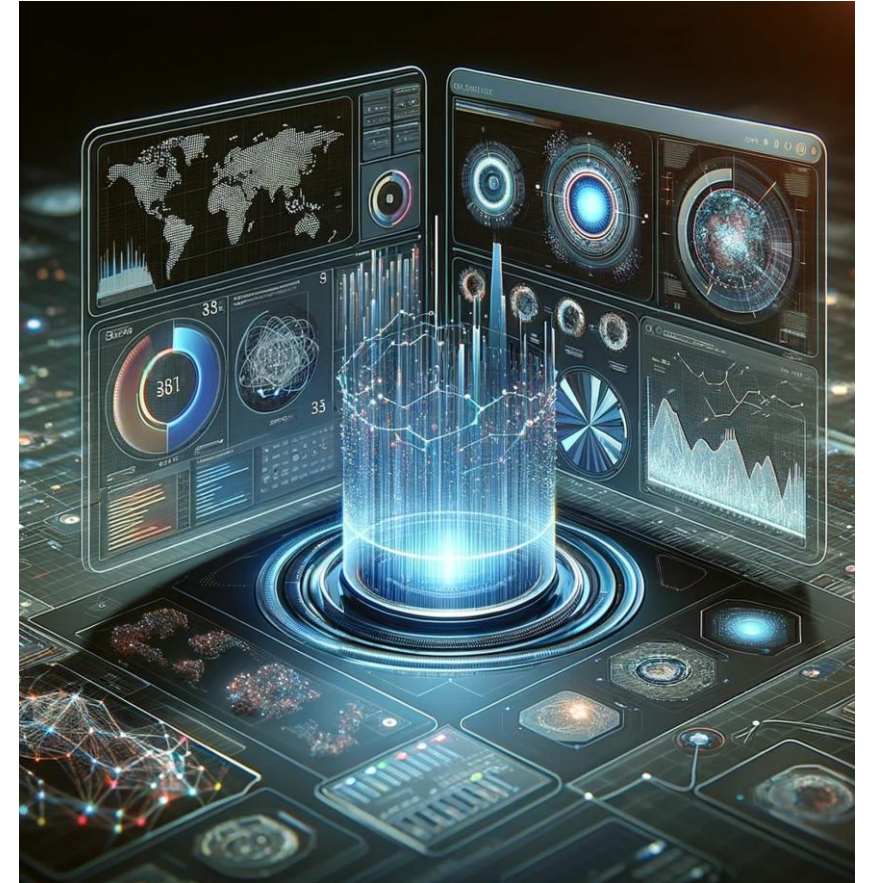
SQL analítico, estatística e machine learning aplicados sobre dados já tratados e documentados.



Entrega

Painéis de BI, APIs, feature stores e relatórios: o insight só vale quando chega a quem decide, no tempo da decisão.

Clareza acima de sofisticação: um gráfico simples que muda uma decisão vale mais que um painel completo que ninguém abre.



On-premises × nuvem: critérios, não preferências



On-premises

VANTAGENS

- Controle total do ambiente e dos dados
- Latência previsível na rede local
- Custo estável em carga constante e alta
- Atende exigências rígidas de soberania

LIMITES

- CAPEX alto e ciclo de compra longo
- Equipe própria para manutenção e plantão
- Escalar exige comprar e instalar hardware



Nuvem

VANTAGENS

- Elasticidade sob demanda, no modelo OPEX
- Serviços gerenciados aceleram a entrega
- Alcance global e recuperação de desastres
- Acesso a hardware especializado sem compra

LIMITES

- Custo recorrente que cresce com o uso
- Dependência de conectividade e do fornecedor
- Governança e soberania exigem configuração

Na prática predominam modelos híbridos. Critérios de decisão: perfil da carga (constante × sazonal), maturidade do time, exigência regulatória e horizonte de amortização do investimento.

IaaS, PaaS e SaaS: quem gerencia o quê

	On-premises	IaaS	PaaS	SaaS
Aplicação	você	você	você	provedor
Dados	você	você	você	provedor
Runtime e bibliotecas	você	você	provedor	provedor
Sistema operacional	você	você	provedor	provedor
Virtualização	você	provedor	provedor	provedor
Servidores e armazenamento	você	provedor	provedor	provedor
Rede e datacenter	você	provedor	provedor	provedor

 Sua equipe gerencia

 O provedor gerencia



Atenção: mesmo em SaaS, a responsabilidade legal pelos dados pessoais continua do controlador (LGPD).

Os três modelos aplicados a projetos de dados



IaaS

Infraestrutura como Serviço

O QUE VOCÊ ALUGA

Máquina virtual, rede e disco — você instala o resto

EXEMPLOS

EC2, Compute Engine, Azure VM

EM DADOS

Cluster Spark instalado, configurado e mantido por você

ESCOLHA QUANDO

Precisa de controle fino, versão específica ou migração "como está"



PaaS

Plataforma como Serviço

O QUE VOCÊ ALUGA

A plataforma pronta: você entrega código e dados

EXEMPLOS

EMR, Dataproc, Databricks, BigQuery, Kafka gerenciado

EM DADOS

Executa o job sem administrar sistema operacional nem patches

ESCOLHA QUANDO

Quer velocidade de entrega com equipe enxuta



SaaS

Software como Serviço

O QUE VOCÊ ALUGA

O software pronto, pela interface ou API

EXEMPLOS

Power BI, Looker Studio, Salesforce, Workspace

EM DADOS

Painel e relatório sem nenhuma infraestrutura própria

ESCOLHA QUANDO

O problema já tem solução de prateleira madura

Serverless atravessa os três modelos: você paga por execução (função, consulta ou job) e o dimensionamento deixa de ser problema seu — mas o custo passa a variar com o uso, e não com a capacidade contratada.

A rede: o gargalo invisível



Shuffle

Agrupamentos e junções no Spark redistribuem dados entre todos os nós. É a fase mais sensível à rede e a primeira suspeita quando um job "trava" perto do fim.



Replicação e ingestão

Cada bloco gravado no HDFS trafega para outros nós; pipelines de streaming mantêm tráfego constante, sem picos previsíveis para manutenção.



Custo de saída (egress)

Na nuvem, entrada costuma ser gratuita e saída é cobrada por GB. Mover dados entre regiões ou provedores pesa mais na fatura do que o processamento.

O QUE MEDIR ANTES DE CULPAR O CÓDIGO

Largura de banda

Quantos GB/s o enlace entrega entre os nós

Latência

Milissegundos de ida e volta; domina cargas com muitas mensagens pequenas

Topologia

Distribuição por rack: replicar em racks distintos protege contra falha física

Segurança, governança e LGPD



Controle de acesso

IAM e RBAC com menor privilégio: cada pessoa e cada serviço acessa só o necessário, com revisão periódica.



Criptografia

Em repouso e em trânsito (TLS), com gestão de chaves separada do armazenamento dos dados.



Catálogo e linhagem

Onde o dado está, o que cada campo significa, de onde veio e quais painéis dependem dele.



Qualidade

Testes automáticos de esquema, nulos, duplicidade e faixas de valor antes de publicar a tabela.



LGPD

Base legal e finalidade declaradas, minimização, atendimento aos direitos do titular e registro do tratamento.



Anonimização

Pseudonimizar, agregar ou mascarar dados pessoais antes de expô-los a análises e treinos de modelo.

A responsabilidade pelo dado pessoal é do controlador: contratar nuvem ou SaaS não transfere o dever legal ao fornecedor — transfere apenas a operação técnica.

Custos: dimensionar antes de contratar

COMO A FATURA É FORMADA

Computação	Por segundo ou hora de máquina ligada, ou por consulta executada
Armazenamento	Por GB-mês, variando conforme a camada (quente, fria, arquivo)
Requisições	Por milhão de operações de leitura, escrita e listagem
Rede	Por GB de saída entre regiões, zonas ou para fora do provedor

ALAVANCAS DE ECONOMIA

- Formato colunar e partições: menos bytes lidos por consulta
- Ciclo de vida por idade do dado: quente, frio, arquivo
- Autoescala e desligamento de clusters ociosos
- Instâncias spot para jobs que toleram interrupção
- Orçamento, marcação por projeto (tagging) e alertas de gasto

8×

Ordem de grandeza da diferença entre varrer 10 TB de CSV e 1,2 TB do mesmo conjunto em Parquet particionado: mesma consulta, mesma resposta, uma fração dos bytes lidos — e do custo, quando a cobrança é por dado processado.

Como escolher: sete perguntas antes de começar

1

Qual é o volume hoje e qual será em 24 meses?

2

Que latência o negócio realmente exige: dia, minuto ou segundo?

3

Que tipo de dado predomina: tabular, texto, mídia ou eventos?

4

Que restrições legais e de soberania se aplicam (LGPD, setor regulado)?

5

Qual é o tamanho e a maturidade do time que vai operar isso?

6

O orçamento é CAPEX ou OPEX, e em que horizonte se amortiza?

7

Quanto custa uma hora de indisponibilidade e quanto custa perder um dado?

Não existe arquitetura melhor — existe a arquitetura adequada a um conjunto de restrições. Documente o porquê de cada escolha: é o que permite revisá-la quando as restrições mudarem.

Síntese e próximos passos

O QUE LEVAR DESTA AULA

- Big Data é um problema de restrições — volume, latência, variedade, confiança e valor.
- A plataforma tem cinco camadas e quatro preocupações transversais.
- A latência exigida define batch ou streaming, e é ela que define o custo.
- Formato e particionamento decidem quanto dado será lido — e, portanto, quanto se paga.
- On-premises, IaaS, PaaS e SaaS diferem por onde passa a fronteira de responsabilidade.
- Privacidade e custo se projetam na arquitetura, não se corrigem depois.

PARA APROFUNDAR

- KLEPPMANN, M. Designing Data-Intensive Applications. O'Reilly, 2017.
- REIS, J.; HOUSLEY, M. Fundamentals of Data Engineering. O'Reilly, 2022.
- WHITE, T. Hadoop: The Definitive Guide. 4. ed. O'Reilly, 2015.
- CHAMBERS, B.; ZAHARIA, M. Spark: The Definitive Guide. O'Reilly, 2018.
- BRASIL. Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais.
- Documentação oficial: Apache Spark, Apache Kafka, Delta Lake e Apache Iceberg.